

14장 부품 열세 개를 이어 붙이면 어디가 터지는가

13주차 · 공공 AI 통합 사례

1장부터 13장까지 부품을 하나씩 만들었다. 이 장은 그 부품을 하나의 운영 흐름으로 다시 조립해, 민원이 실시간으로 들어오고 하루가 마감되면 집계가 확정되며 그 확정치가 다음 날 예측의 입력이 되는 작은 시스템을 돌린다. 새 기술은 하나도 나오지 않는 대신, 부품과 부품이 만나는 자리에서만 생기는 고장을 일부러 일으켜 본다.

1. 오늘의 질문

- 부품을 옮겨 붙이는 동안 값이 변하지 않았다는 것을 무엇으로 증명할까?
- 같은 민원이 두 번 들어왔는데 집계가 그대로라면, 그 사실을 어떻게 보일까?
- 모델 API가 죽은 순간 하루 마감은 어떻게 되어야 할까?
- 앞 장들이 각자 정한 값을 한 시스템에 모으면 왜 서로 부딪칠까?

2. 완성차 출고 검사로 보는 이 장

부품을 따로 시험한 기록이 있어도 조립한 차를 그대로 내보내지는 않는다. 조립 후에 다시 재고, 일부러 급제동을 걸어 보고, 항목마다 합격 여부를 적은 검사표를 붙여 내보낸다. 이 장이 하는 일이 그것이다.

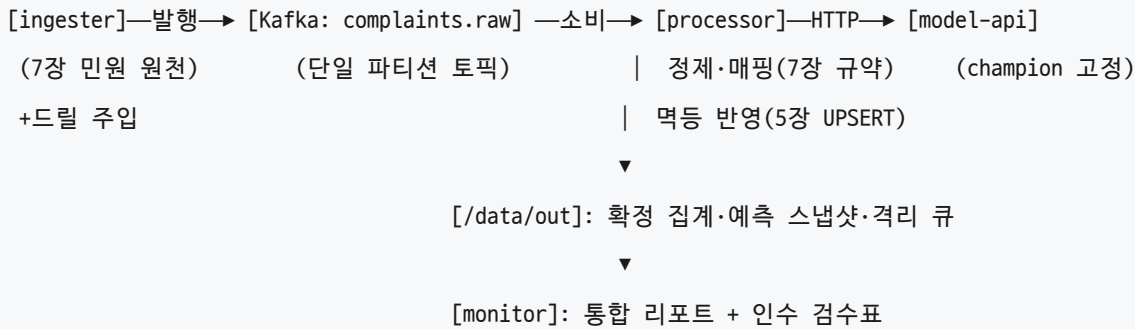
완성차 출고 검사	이 장의 통합 시스템
부품마다 따로 남겨 둔 규격 시험 기록	4~11장 실습이 각각 커밋해 둔 실측값
조립한 뒤 부품 번호를 원장과 다시 대조	서빙 앱 해시(hash)·피쳐 지문·확정치 재대조
급제동·충돌 시험을 일부러 한다	드릴(drill) 3종 — 오염·중복·의존 서비스 중단
항목마다 합격 여부를 적은 출고 검사표	인수 검수표 16항목(모두 자동 계산)

비유가 깨지는 지점: 자동차 부품은 눈으로 보면 같은 부품인지 알 수 있지만, 복사된 파일은 이름이 같아도 내용이 다를 수 있다. 그래서 이 장은 "같은 코드로 만들었다"가 아니라 **바이트가 같다**를 해시로 확인한다.

3. 통합은 재발명이 아니라 재조립이다

앞 장의 무엇이 어디에 다시 놓이는가

통합 MVP(Minimum Viable Product, 핵심 경로만 넣은 최소 운영 시스템)의 구조는 다음과 같다. 모든 연결은 Docker Compose 내부 네트워크 안에 있고, 바깥에 열리는 것은 모델 API 포트 하나뿐이다.



부품마다 원래 장이 있다. 메시지 버스는 4장의 Kafka이고, 중복 방지는 5장의 `complaint_id` 업서트(UPSERT)이며, 정제와 표준코드 매핑과 `_SUCCESS` 마커(marker)는 7장의 규약이다. 피처는 8장의 자리인데 여기서는 Feast 대신 확정 집계를 그대로 쓰고, 모델은 9장이 승격한 champion을, 서버는 10장 `app.py`를 바이트 그대로 복사해 쓰며, 지연 채점은 11장의 감시 경로다. 14장이 새로 만든 규칙은 하나도 없고, 한 일은 이 부품들을 이어 붙인 것뿐이다.

하루의 흐름은 두 개의 시계로 돌아간다. 접수는 스트림(stream)의 시계를 따라 이벤트가 오는 대로 정제·반영되고, 확정과 예측은 배치의 시계를 따라 하루가 마감된 뒤에야 나간다. 마감이 오면 processor는 그날의 집계를 확정하고, 어제 만든 예측을 오늘 도착한 실제 건수로 채점한 다음, 오늘 확정 건수를 입력으로 내일 예측을 만든다. **확정 → 채점 → 예측**이라는 이 세 박자가 매일 반복된다.

옳겨 붙이는 동안 값이 변하지 않았다는 증명

통합에서 가장 흔한 의문은 "옳겨 붙이다 무엇인가 달라지지 않았나"이다. 이 실습은 그 질문에 말이 아니라 값으로 답한다. 앞 장들이 커밋해 둔 실측과 이 장의 실행 결과를 코드가 자동 대조하고, 그 결과가 인수 검수표의 항목이 된다.

- **서빙 앱 동일성** — 10장 `app.py`의 sha256 `ec7ec653...`과 복사본의 해시가 일치한다(검수표 14항).
- **훈련 데이터 동일성** — 9장 피처 지문 `96f6b6b3...`과 bootstrap이 재구성한 데이터의 지문이 일치한다(15항).
- **집계 재구현** — 7장이 배치로 낸 3일치 확정치와 이 장이 스트림으로 낸 확정치가 같다(02항).
- **예측 동일성** — champion 호출 9건이 모두 6.0을 돌려준다(07항).
- **평가 동일성** — 채점된 예측 6건의 전체 MAE가 9장 훈련 MAE와 같은 1.0이다(13항).

셋째 항목이 이 장에서 가장 중요하다. 7장은 파일을 읽는 배치로 집계했고 이 장은 Kafka를 소비하는 스트림으로 집계했는데, 경로가 전혀 다른데도 확정치가 같다. 처리 방식이 바뀌어도 정제 규칙·먹등 키·집계 기준이라는 업무 규약이 같으면 결과가 같아야 하고, 그 "같아야 한다"를 수치 대조로 증명할 수 있다는 것이 이 검증의 요지다.

다섯째 항목은 계산상 필연이다. 채점된 6건은 9장이 학습에 쓴 쌍과 같은 데이터이고 모델도 같으므로 $(1+1+0+0+1+3)/6 = 1.0$ 이 나올 수밖에 없다. 값을 맞히려는 것이 아니라 감시 경로 자체가 제대로 계산하고 있는지 확인하는 절차다.

인수 검수표 16항목은 전부 이런 성질이다. 어느 항목도 "잘 되었다"는 사람의 주장이 아니라, 실행 증거에서 계산된 참·거짓이다. 예측 자체는 아무것도 집행하지 않고 기록으로만 남으며, 자원을 옮길지 안내를 보낼지는 사람이 판단한다.

4. 중복은 지워지는 것이 아니라 흡수된다

왜 중복이 들어오는가

4장에서 본 대로 메시지 전송은 대개 at-least-once로 설계된다. 보낸 쪽이 확인 응답을 받지 못하면 다시 보내므로, 유실을 막는 대신 중복이 생긴다. 여기에 원천 자체의 중복도 섞인다. 민원인이 같은 건을 두 번 접수하면 재전송이 아닌데도 같은 식별자가 두 번 도착한다.

드릴 D2는 이 상황을 그대로 재현한다. 7/2의 이벤트 18건을 발행한 뒤 마지막 10건을 한 번 더 발행했다. 실측은 물리 수신 28건, 고유 17건, 중복 흡수 11건이었고, 매핑 집계는 16건으로 7장 확정치와 같았다.

흡수된 11건의 내역

흡수된 11건은 성격이 다른 둘로 나뉜다. 수송 계층의 재전송이 10건이고, 업무 계층의 중복 접수가 1건이다. 원인은 다르지만 소비자 쪽에서 보이는 모습은 똑같다. 같은 `complaint_id`가 다시 도착한 것이다.

5장의 먹등(idempotent) 키가 강력한 이유가 여기 있다. 업무 식별자를 키로 삼아 업서트하면 원인을 구분할 필요 없이 둘 다 무해해지고, 관찰 횟수만 늘어난 흔적으로 남는다. 만약 두 계층에 각각 다른 중복 제거 장치를 두었다면, 어느 쪽이 어느 중복을 맡는지부터 정해야 했을 것이다.

이벤트 하나를 끝까지 따라가면 이 흡수가 눈에 보인다. 7/2의 `C260702-016`은 재전송 대상 10건에 포함되어 두 번 발행되었는데, 첫 수신에서 "관악"이 "관악구"로 보정되어 표준코드 11620에 매핑된 뒤 삽입되었고, 두 번째 도착에서는 업무 상태가 그대로인 채 관찰 횟수만 2가 되었다. 같은 날 `C260702-001`은 재전송이 아니라 원천 파일 안에서 두 번 접수된 건인데, 소비자 쪽 기록은 똑같이 관찰 횟수 2다. 원인이 다른 두 중복이 같은 장치에 같은 모습으로 흡수된 것이다.

같은 원칙이 소비자의 반대편에도 걸려 있다. processor는 오프셋(offset) 자동 커밋을 끄고, 메시지를 파일과 상태 DB에 다 쓴 뒤에야 커밋한다. 자동 커밋은 처리가 끝나기 전에 오프셋을 밀어 올릴 수 있어, 그 사이에 프로그램이 멈추면 그 메시지가 다시 재생되지 않기 때문이다. 늦게 커밋하면 재생이 생기지만, 그

재생을 무해하게 만드는 것이 바로 위의 맥등 설계다. 전달 보증과 맥등은 별개 기능이 아니라 한 쌍이다.

회계 보존식이 하는 일

집계가 변하지 않았다는 말만으로는 부족하다. 중복이 흡수된 것인지 애초에 없었던 것인지 구분되지 않기 때문이다. 그래서 매일 두 개의 등식을 기록한다.

물리 28 = 고유 17 + 흡수 11이고, 고유 17 = 매핑 16 + 미기재 0 + 미매핑 1이다. 왼쪽과 오른쪽이 맞아떨어지면 사라진 건이 없다는 뜻이고, 어긋나면 어느 갈래에서 조용히 없어졌다는 뜻이다. 이 등식이 성립하는지가 검수표 04항이고, 물리 28건이 들어와도 매핑 16건이 변하지 않았다는 것이 05항이다.

여기서 배울 것은 "중복이 없었다"가 아니다. **중복이 분명히 있었는데도 값이 같다는 것을 회계로 증명할 수 있다는 점**이다.

5. 실패는 가장 작은 단위로 잘라 값으로 남긴다

D1 — 오염 이벤트 한 건이 전체를 멈추지 않는다

7/3의 정상 이벤트 사이에 JSON으로 읽히지 않는 페이로드 1건을 끼워 발행했다. 맨 앞이나 맨 뒤가 아니라 한복판에 넣은 이유가 있다. 파이프라인이 "그 자리에서" 멈추지 않고 다음 메시지로 계속 가는지를 봐야 하기 때문이다. processor는 그 메시지를 격리 큐(quarantine queue) 파일에 원문과 사유(json_parse_error) 짝 보존하고 넘어갔고, 격리 1건이 남았으며 7/3 매핑 집계는 20건으로 온전했고 마감도 정상 확정되었다.

격리 큐가 없으면 선택지는 둘뿐이다. 파이프라인이 그 자리에서 멈추거나, 오염이 집계에 섞여 들어가거나다. 앞은 가용성을 잃고 뒤는 정확성을 잃는다. 2장에서 세운 원칙이 셋째 길이다. **잘못된 데이터는 버리지도 통과시키지도 않고 분리해서 보존한다.** 보존된 원문은 나중에 상류의 버그인지 공격인지 가려내고 수리 후 재처리하는 재료가 된다.

D3 — 마감은 살고 예측만 대기한다

가장 중요한 드릴이다. 7/3 마감 직전에 모델 API 컨테이너(container)를 정지시키고 그대로 마감을 진행했다. 결과는 이렇게 걸렸다. 집계 확정과 어제 예측의 채점은 모델이 필요 없는 일이라 완료되었고, 모델이 필요한 내일 예측 3건만 pending_retry 상태로 사유(api_unreachable)와 함께 기록되었다. _SUCCESS 마커는 정상적으로 찍혔다.

이것이 **부분 실패 격리**다. "모델 API가 죽으면 마감 전체가 실패"가 아니라, 실패를 가장 작은 단위인 예측 3건으로 잘라 상태로 기록하고 나머지는 확정한다. 9장에서 게이트 통과 여부와 승격 여부를 다른 필드로 나눈 것, 7장에서 실패를 마커의 부재로 남긴 것과 같은 원칙이 세 번째로 나온 자리다.

복구는 두 단계였다. 컨테이너를 다시 띄워 `/health` 가 정상 응답하는지 확인한 뒤 재시도 제어 이벤트를 발행하자, 대기 3건이 모두 2회째 시도에서 성공해 대기 0건으로 수렴했다. 재시도가 안전했던 이유는 복구 코드가 아니라 평시의 설계에 있다. 예측 기록이 (대상일, 지역)을 키로 하는 멍등 쓰기라서, 재시도가 겹쳐도 중복 기록이 생기지 않는다.

격리해 두면 새벽 2시가 달라진다

이 설계가 무엇을 했는지는 장애가 난 뒤에 드러난다. 감시 리포트에 대기 3건이 찍혀 있을 때, 당직자가 하는 일은 원인을 추측하는 것이 아니라 `/health` 를 한 번 호출해 보는 것이다. 응답이 없으면 컨테이너를 다시 띄우고 재시도를 발행한 뒤 대기가 0건이 되는지 확인하면 끝난다.

여기서 더 중요한 것은 **확인하지 않아도 되는 것**이 무엇인지 미리 정해져 있다는 점이다. 오늘 집계 손상되었는지는 확인할 필요가 없다. `_SUCCESS` 가 있으므로 확정본이다. 예측이 유실되었는지도 확인할 필요가 없다. 대기 3건이 입력값째 상태로 남아 있다. 격리가 없었다면 이 두 가지부터 새벽에 뒤져야 했을 것이다.

세 드릴을 관통하는 질문은 하나다. **실패를 어디에 둘 것인가**. 오염은 격리 큐에, 중복은 멍등 키에, 의존 서비스의 중단은 `pending_retry` 상태에 둔다. 셋 다 실패를 없애지 않고 값으로 남기며, 값으로 남아 있어야 나중에 세고 감사할 수 있다. 격리 큐에 쌓인 건수가 늘면 상류의 형식이 바뀐 것이고, 그 건들이 오래 머물면 재처리 절차가 돌지 않는 것이라는 판독도 값이 남아 있어야 가능하다.

6. 계층별 결정은 서로 모순되지 않아야 한다

통합에서 실제로 터지는 것은 부품의 결함이 아니라 계층별 결정의 불일치다. 앞 장들은 각자의 맥락에서 값을 하나씩 정했다. 4장은 파티션(partition) 수를, 5장은 중복 제거 키를, 7장은 배치 주기와 마커 규약을, 9장은 승격 게이트를, 11장은 감시 임계와 판정 주기를 정했다. 각 값은 자기 장 안에서 근거가 분명했지만, 한 시스템에 모으면 서로를 무효로 만드는 조합이 나온다.

이 MVP에서 실제로 걸린 조합이 셋이다.

파티션 수 대 마감 순서. 4장은 처리량을 위해 파티션 3개를 권했는데, 7장의 일 마감은 그날 이벤트가 전부 도착한 뒤에 마감 신호가 온다는 것을 전제한다. 파티션이 여럿이면 그 전제가 깨지므로 이 MVP는 파티션을 1개로 줄였다. 처리량을 마감 순서와 맞바꾼 것이고, 그 맞바꿈을 범위표에 기록해 둔다.

게이트 값 대 표본 크기. 9장의 승격 게이트는 MAE 1.05인데 11장 방식의 지연 채점은 하루 표본이 3건이다. 표본 3건짜리 MAE로 게이트를 판정하면 하루의 우연이 재학습을 부르는데, 7/3의 1.3333이 정확히 그 경우다. 게이트 값을 바꾸거나 판정 주기를 늘리거나 하나는 조정해야 하며, 이 MVP는 후자를 택해 그 값을 감시 강화 신호로만 읽는다.

두 층의 멍등 키. 5장의 멍등 키는 `complaint_id` 이고 이 장의 예측 재시도 키는 (대상일, 지역)이다. 서로 다른 층의 결정인데 둘 다 멍등이라서 D3의 재시도가 안전했다. 한쪽만 멍등이 아니어도 복구 절차 자체가 새 오염을 만들었을 것이다.

세 조합의 형태는 같다. **한 계층의 최적값이 다른 계층의 전제를 깬다.** 그래서 통합 설계의 첫 작업은 새 값을 정하는 것이 아니라, 이미 정해진 값을 한 표에 모아 충돌을 찾는 것이다.

충돌이 나오지 않는 칸도 있다. 13장의 접근 등급이 그런데, 이 MVP는 피처가 한 종이고 그 피처를 쓰는 모델도 하나여서 접근 규약이 암묵으로 남았기 때문이다. 그래서 다른 어느 결정과도 부딪치지 않는다. 뒤에 나올 점검표에서 그 칸을 비우지 말고 "왜 충돌이 없는지"를 적게 하는 이유가 이것이다. 충돌이 없다는 판단에도 조건이 붙어 있고, 피처가 늘어나는 순간 그 조건이 사라진다.

이 점검은 요구가 바뀔 때마다 다시 해야 한다. 같은 부품으로 재난 대응을 만든다고 하면 침수 신고 집계 하루 마감을 기다릴 수 없으므로 6장의 분 단위 창과 워터마크가 들어오고, 그 순간 "일 마감이 그날 이벤트를 전부 받은 뒤에 온다"는 전제 위에 서 있던 단일 파티션 결정이 근거를 잃는다. 부품은 그대로인데 시계가 바뀌면 조합이 통째로 달라진다.

정해진 값을 한 표에 모으는 일이 왜 문서 작업이 아닌지도 여기서 분명해진다. 파티션 수와 게이트 값은 실행 로그 어디에도 "이 둘이 서로 부딪칩니다"라고 찍히지 않는다. 로그는 그날 마감이 되었는지, MAE가 얼마였는지만 말해 줄 뿐이고, 그 값이 다른 계층의 전제를 깨고 있는지는 표를 만들어 사람이 맞대어 봐야 보인다.

7. 실습 14.1 Docker Compose 기반 통합 MVP 실행 (난이도

★★★★☆)

실습 목표

- 앞 장들이 정한 계층별 결정을 한 표로 모아 충돌 지점을 먼저 찾는다(설계).
- 통합 MVP를 기동해 3일치 운영과 드릴 3종을 완주하고, 예측했던 것과 실측을 맞춰 본다(대조).
- 재조립 교차 검증과 인수 검수표 16항목을 전항 통과시킨다(검수).

1단계 설계 — 계층 결정 일관성 점검

실행하기 **전에** 아래 표를 채운다. 값 자체는 앞 장 실습에서 이미 정했으므로 새로 만들 것이 없다. 이 표가 묻는 것은 그 값들을 한 시스템에 모았을 때 서로 버티는가다.

계층	원 장이 정한 값	이 통합에서 쓸 값	왜 그 값인가	어느 결정과 충돌할 수 있는가
토픽·파티션 수(4장)				
중복 제거 키(5장)				
창·마감 방식(6·7장)				
배치 주기(7장)				
승격 게이트(9장)				
감시 임계·판정 주기 (11장)				
접근 등급(13장)				
이 조합이 깨지는 조건				

작성 규칙이 셋이다. 넷째 열에 "앞 장에서 그렇게 정했으니까"를 쓰지 않고 다른 결정과 어떻게 맞물리는지를 적는다. 다섯째 열은 한 칸도 비우지 않으며, 충돌 후보가 없다고 보면 왜 없는지를 적는다. 마지막 행은 실행 뒤 3단계에서 다시 쓴다.

이어서 드릴 예측을 적는다. D1에서 오염 이벤트 1건을 넣으면, D2에서 재전송 10건을 넣으면, D3에서 마감 직전 모델 API를 정지시키면 각각 어느 계층이 먼저 무너질 것 같은지와 그렇게 본 근거를 쓴다. "아무것도 무너지지 않는다"도 답으로 인정하되, 무엇이 그것을 막는지를 근거에 적는다.

실행 환경

Python 3.10 이상(3.13에서 검증), Docker Desktop 실행 상태. mlflow 3.14.0, scikit-learn 1.9.0, kafka-python 3.0.7.

시작 전 준비

이 실습은 **Docker Desktop**이 실행 중이어야 한다. 통합 MVP를 컨테이너로 띄워 드릴을 돌리기 때문이다.

```
python scripts/setup_practice.py 14
```

실행 명령

```
cd practice/chapter14
python3 -m venv venv && source venv/bin/activate
pip install -r code/requirements.txt
python run_chapter14.py
```

핵심 코드

부분 실패 격리 — 마감(확정 가능한 일)과 예측(재시도 가능한 일)의 분리:

```
resp, err, ms = call_predict(r["lawd_cd"], r["count"])
if resp is None:                                # 모델 API 실패 - 마감은 계속 간다
    row = (target, r["lawd_cd"], ..., "pending_retry", 1, err) # 실패를 값으로 기록
(out / "_SUCCESS").write_text("")               # 마커는 마지막에(7장) - 집계는 확정된다
```

전체 코드는 *practice/chapter14/code/14-1-integrated-mvp/* 참고

2단계 실행 결과 확인 사항

1. **bootstrap**: 지문 96f6b6b3... 출력 후 CH14_BOOTSTRAP_OK 로 종료(9장 스냅샷 일치)
2. **서빙 앱 동일성**: sha256 일치 출력(10장 app.py 와 바이트 동일)
3. **스모크**: predict(전일 9) → 6.0, 잘못된 입력에는 422
4. **2일차 마감**: 물리수신 28 / 고유 17 (중복 흡수 11) → 매핑 16 (드릴 D2)
5. **드릴 D3**: 마감 확정(_SUCCESS) + 예측 대기 3건 뒤 복구 수렴: 대기 0건
6. **인수 검수표**: 16항목 전부 [PASS], 마지막 줄 CH14_RUN_PASS

3일 운영 시뮬레이션 실측

항목	7/1	7/2 (드릴 D2)	7/3 (드릴 D1-D3)
물리 수신	20	28	22
고유(먹등 흡수 후)	20	17	22
중복 흡수	0	11(재전송 10 + 중복 접수 1)	0
매핑/미기재/미매핑	18/1/1	16/0/1	20/1/1
확정 집계(마포·관악·강남)	5-5-8	5-5-6	6-5-9
익일 예측(3개 구)	전부 6.0	전부 6.0	중단 → 대기 3건 → 복구 후 6.0
전일 예측의 채점(일 MAE)	—	0.6667	1.3333
격리(오염)	0	0	1

3단계 대조 — 예측과 실측 맞춰 보기

1단계에서 적어 둔 드릴 예측표의 "실제" 칸을 위 표의 값으로 채운다. 예측이 어긋나기 쉬운 자리가 셋이다.

D1을 집계 오차로 예측했다면 격리 큐를 셈에 넣지 않은 것이다. 7/3 매핑은 20건으로 온전했고 오염 1건은 집계 밖 파일에 남았다. 오염이 집계를 흔드는 것은 격리 큐가 없을 때뿐이다.

D2를 집계 증가로 예측했다면 물리 수신과 고유를 같은 것으로 본 것이다. 실측은 물리 28 → 고유 17 → 매핑 16이고 7장 확정치와 같으며, 늘어난 것은 수신 건수뿐이다.

D3를 마감 실패로 예측했다면 그 예측이 오히려 대부분의 시스템에서 맞다. 여기서 마감이 살아남은 것은 운이 아니라 확정 가능한 일과 모델이 필요한 일을 미리 나눠 둔 설계 때문이며, 예측이 빗나간 그 지점이 곧 설계가 한 일이다.

끝으로 1단계 표의 마지막 행("이 조합이 깨지는 조건")을 다시 쓴다. 파티션을 3개로 올리면 일 마감이 어느 조건에서 틀리는지, 표본 3건짜리 MAE로 게이트를 판정하면 7/3의 1.3333이 무엇을 부르는지를 실측 옆에 적는다. 제출본은 이 수정을 거친 최종본이다.

8. 핵심 요약

1. 재조립이 원본과 같은 값을 낸다는 것을 값으로 증명한다. 서빙 앱 해시(`ec7ec653...`), 피쳐 지문(`96f6b6b3...`), 7장 확정치, 예측 6.0, 전체 MAE 1.0 다섯 가지를 코드가 자동 대조한다. 특히 배치로 집계한 7장과 스트림으로 집계한 이 장이 같은 확정치를 낸 것은, 처리 방식이 아니라 업무 규약이 결과를 정한다는 뜻이다.

2. 중복은 지워지는 것이 아니라 흡수되며, 흡수는 회계로 증명한다. 7/2에 물리 28건이 들어와 고유 17건이 되고 매핑 16건으로 확정되었다. 흡수된 11건은 수송 계층의 재전송 10건과 업무 계층의 중복 접수 1건인데, 원인은 달라도 같은 먹등 키 하나가 둘 다 무해하게 만든다.

3. 실패는 가장 작은 단위로 잘라 값으로 남긴다. 모델 API가 죽은 7/3에도 집계는 확정되고 어제 예측은 채점되었으며, 모델이 필요한 예측 3건만 `pending_retry` 로 남았다가 재시도 후 0건으로 수렴했다. 재시도가 안전했던 것은 복구 코드가 아니라 (대상일, 지역) 맥등 쓰기 덕분이다.

4. 통합 설계의 첫 작업은 값을 정하는 것이 아니라 충돌을 찾는 것이다. 파티션 3개는 일 마감의 전제를 깨고, 게이트 1.05는 하루 표본 3건과 맞지 않는다. 각 값이 자기 장 안에서 옳았다는 사실이 한 시스템 안에서도 옳다는 것을 보장하지 않는다.

5. 인수 검수표는 주장이 아니라 계산이다. 16항목 전부가 실행 증거에서 나온 참·거짓이며, 사람이 "잘 됩니다"라고 말할 자리가 없다. 예측 자체는 아무것도 집행하지 않고 기록으로만 남고, 자원을 옮길지는 사람이 판단한다.

9. 과제

실습 14.1을 실행한 뒤 다음 세 가지를 LMS에 업로드한다.

1. 계층 결정 일관성 점검표(3단계 수정을 반영한 최종본) — 텍스트·이미지 어느 형식이든 무방
2. `practice/chapter14/data/output/ch14_acceptance_checklist.json` — 생성된 파일 그대로
3. 드릴 예측·실측 대조표(1단계 예측과 3단계 실제 칸이 모두 채워진 것)

이어서 본인 검수표를 근거로 두 가지를 각각 세 문장 이내로 쓴다.

1. 검수표에서 피쳐 지문 일치와 서빙 앱 바이트 동일을 확인하는 항목을 찾아 값을 적고, 두 항목이 각각 무엇을 보증하는지 쓴다.
2. 배치 경로(7장)와 스트림 경로(14장)가 같은 확정치를 낸 이유를 정제·맥등·집계·마감 중에서 골라 설명한다.

점검표는 값의 정답 여부가 아니라 넷째·다섯째 열이 채워져 있는지를 본다. 수업일 포함 7일 이내에 제출한다.

10. 더 알아보기

이 장에서 다루지 않은 내용은 실무교재 `docs/chapter14.md` 에 있다.

- **요구사항 정의와 "실시간"의 계약 언어** — 기능·비기능 요구를 나누고 각각에 검증 방법을 붙이는 표: §14.1
- **정책 의사결정 지원의 경계** — 자동 집행 금지 지점, 사람 개입 지점, 예측 오차 평가와 정책효과 평가의 구분: §14.3
- **MVP 범위표와 운영 runbook** — 무엇을 왜 뺐고 언제 다시 넣는지, 증상에서 출발하는 장애 대응 절차서: §14.4, §14.5
- **검수표 16항목 전체와 산출물 목록** — 항목별 자동 대조 근거: §14.6, 그리고 `practice/chapter14/data/output/`